

**DEPARTMENT OF COMPUTER SCIENCE
HUMAN-COMPUTER INTERACTION (HCI) &
COMPUTER GRAPHICS (CG)**

LAB 1 REPORT

Display Density Metrics & Image Array Mechanics in NumPy

Name: Noor Ahmed khan

Roll Number: CSE/2K24/113

1. Lab Objectives

1. Compute spatial display metrics including resolution, total pixels, aspect ratio, and DPI/PPI.
2. Set up a Python computing environment for visual graphics tasks.
3. Construct, inspect, and manipulate 3D image arrays ($H \times W \times C$) using NumPy.
4. Perform color channel extraction, spatial downsampling, and array striding operations.

2. Software / Environment

Python 3 with NumPy, Pillow, and Matplotlib was used to complete the laboratory tasks.

Required packages:

- NumPy
- Pillow
- Matplotlib

3. Task 1 — Display Pixel Density (PPI/DPI) Calculator

The program accepts horizontal resolution, vertical resolution, and physical screen diagonal size. It calculates total pixel count, simplified aspect ratio, DPI/PPI, and classifies the display density.

3.1 Python Source Code

```
import math

Wpx = int(input("Enter horizontal resolution (pixels): "))
Hpx = int(input("Enter vertical resolution (pixels): "))
Dinches = float(input("Enter physical diagonal size (inches): "))

total_pixels = Wpx * Hpx

gcd = math.gcd(Wpx, Hpx)
aspect_width = Wpx // gcd
aspect_height = Hpx // gcd

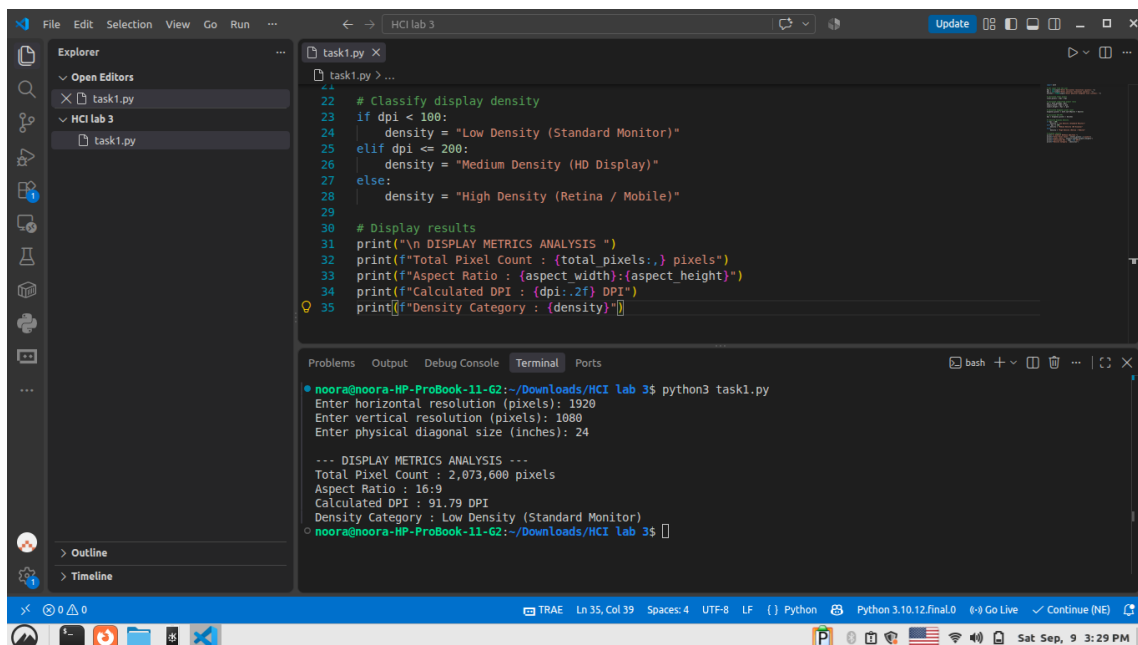
diagonal_pixels = math.sqrt(Wpx**2 + Hpx**2)
dpi = diagonal_pixels / Dinches

if dpi < 100:
    density = "Low Density (Standard Monitor)"
elif dpi <= 200:
    density = "Medium Density (HD Display)"
else:
    density = "High Density (Retina / Mobile)"

print("\n DISPLAY METRICS ANALYSIS ")
print(f"Total Pixel Count : {total_pixels:,} pixels")
print(f"Aspect Ratio : {aspect_width}:{aspect_height}")
print(f"Calculated DPI : {dpi:.2f} DPI")
print(f"Density Category : {density}")
```

3.2 Desktop Monitor Test

Input: 1920 × 1080 pixels, 24 inches



The screenshot shows a Visual Studio Code editor window with a file explorer on the left and a terminal at the bottom. The file explorer shows a project named 'HCI lab 3' containing a file 'task1.py'. The editor displays the Python code for the calculator. The terminal shows the execution of the script, where the user inputs 1920 for horizontal resolution, 1080 for vertical resolution, and 24 for physical diagonal size. The output displays the total pixel count as 2,073,600, the aspect ratio as 16:9, the calculated DPI as 91.79, and the density category as 'Low Density (Standard Monitor)'.

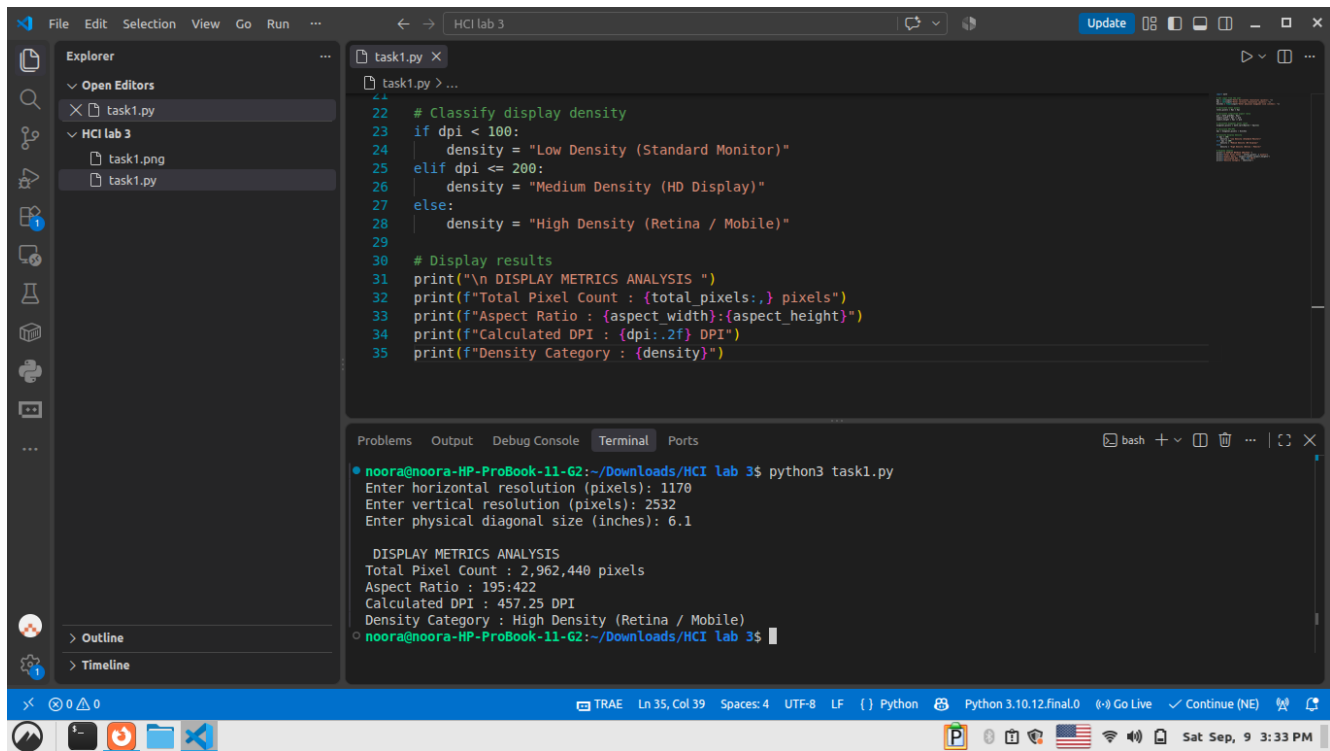
```
task1.py
22 # Classify display density
23 if dpi < 100:
24     density = "Low Density (Standard Monitor)"
25 elif dpi <= 200:
26     density = "Medium Density (HD Display)"
27 else:
28     density = "High Density (Retina / Mobile)"
29
30 # Display results
31 print("\n DISPLAY METRICS ANALYSIS ")
32 print(f"Total Pixel Count : {total_pixels:,} pixels")
33 print(f"Aspect Ratio : {aspect_width}:{aspect_height}")
34 print(f"Calculated DPI : {dpi:.2f} DPI")
35 print(f"Density Category : {density}")

noora@noora-HP-ProBook-11-G2:~/Downloads/HCI lab 3$ python3 task1.py
Enter horizontal resolution (pixels): 1920
Enter vertical resolution (pixels): 1080
Enter physical diagonal size (inches): 24

--- DISPLAY METRICS ANALYSIS ---
Total Pixel Count : 2,073,600 pixels
Aspect Ratio : 16:9
Calculated DPI : 91.79 DPI
Density Category : Low Density (Standard Monitor)
noora@noora-HP-ProBook-11-G2:~/Downloads/HCI lab 3$
```

3.3 Smartphone Test

Input: 1170 × 2532 pixels, 6.1 inches



The screenshot shows a Visual Studio Code editor window with a Python file named `task1.py` open. The file contains a script to classify display density based on horizontal resolution, vertical resolution, and physical diagonal size. The script uses an if-elif-else structure to categorize the display as Low Density (Standard Monitor), Medium Density (HD Display), or High Density (Retina / Mobile). It also prints out the total pixel count, aspect ratio, and calculated DPI.

```
22 # Classify display density
23 if dpi < 100:
24     density = "Low Density (Standard Monitor)"
25 elif dpi <= 200:
26     density = "Medium Density (HD Display)"
27 else:
28     density = "High Density (Retina / Mobile)"
29
30 # Display results
31 print("\n DISPLAY METRICS ANALYSIS ")
32 print(f"Total Pixel Count : {total_pixels:,} pixels")
33 print(f"Aspect Ratio : {aspect_width}:{aspect_height}")
34 print(f"Calculated DPI : {dpi:.2f} DPI")
35 print(f"Density Category : {density}")
```

The terminal output shows the script being executed with the following inputs and results:

```
noora@noora-HP-ProBook-11-G2:~/Downloads/HCI lab 3$ python3 task1.py
Enter horizontal resolution (pixels): 1170
Enter vertical resolution (pixels): 2532
Enter physical diagonal size (inches): 6.1

DISPLAY METRICS ANALYSIS
Total Pixel Count : 2,962,440 pixels
Aspect Ratio : 195:422
Calculated DPI : 457.25 DPI
Density Category : High Density (Retina / Mobile)
noora@noora-HP-ProBook-11-G2:~/Downloads/HCI lab 3$
```

4. Task 2 — Environment Setup & Synthetic Image Matrix Creation

A $300 \times 400 \times 3$ NumPy image matrix was created using `np.zeros()` with `uint8` data type. The four quadrants were filled with red, green, blue, and white.

4.1 Python Source Code

```
import numpy as np
```

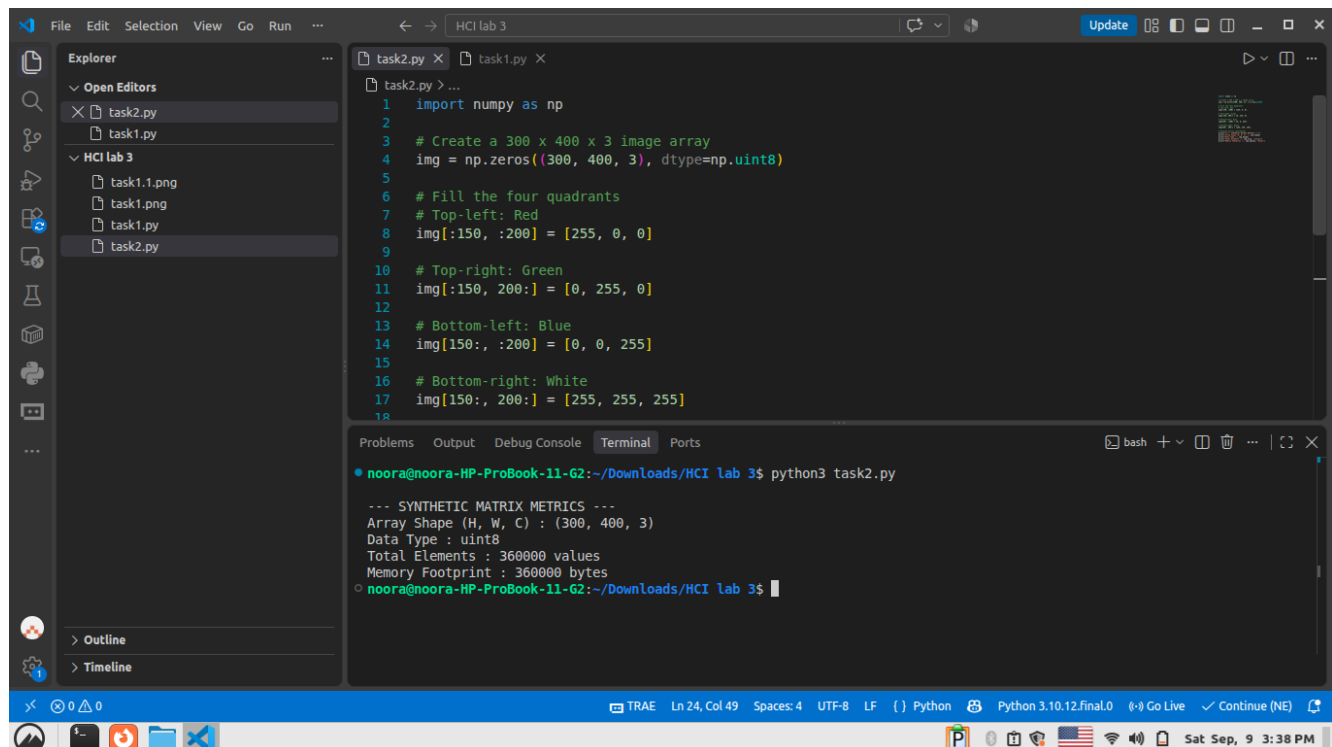
```
img = np.zeros((300, 400, 3), dtype=np.uint8)
```

```
img[:150, :200] = [255, 0, 0]
img[:150, 200:] = [0, 255, 0]
img[150:, :200] = [0, 0, 255]
img[150:, 200:] = [255, 255, 255]
```

```
print("\n--- SYNTHETIC MATRIX METRICS ---")
print("Array Shape (H, W, C) :", img.shape)
print("Data Type :", img.dtype)
print("Total Elements :", img.size, "values")
print("Memory Footprint :", img.nbytes, "bytes")
```

4.2 Output

Expected matrix metrics: (300, 400, 3), `uint8`, 360,000 elements, and 360,000 bytes.



The screenshot shows a Visual Studio Code editor window with a file explorer on the left and a terminal at the bottom. The file explorer shows a project named 'HCI lab 3' containing files 'task1.png', 'task1.py', and 'task2.py'. The editor displays the source code for 'task2.py', which creates a 300x400x3 image matrix and fills its quadrants with red, green, blue, and white. The terminal shows the command 'python3 task2.py' being executed, resulting in the following output:

```
--- SYNTHETIC MATRIX METRICS ---
Array Shape (H, W, C) : (300, 400, 3)
Data Type : uint8
Total Elements : 360000 values
Memory Footprint : 360000 bytes
```

5. Task 3 — Channel Slicing & Isolation

The input image was loaded and the Red, Green, and Blue channels were extracted using Axis 2 slicing. Three isolated color images and three grayscale intensity maps were displayed in a 2×3 subplot layout.

5.1 Python Source Code

```
import numpy as np
from PIL import Image
import matplotlib.pyplot as plt

image = Image.open("image.jpeg").convert("RGB")
img = np.array(image)

red = img[:, :, 0]
green = img[:, :, 1]
blue = img[:, :, 2]

red_only = np.zeros_like(img)
green_only = np.zeros_like(img)
blue_only = np.zeros_like(img)

red_only[:, :, 0] = red
green_only[:, :, 1] = green
blue_only[:, :, 2] = blue

print("\n--- CHANNEL EXTRACTION SUMMARY ---")
print("Original Image Shape :", img.shape)
print(f"Red Channel 2D Shape : {red.shape} | Mean Intensity: {red.mean():.2f}")
print(f"Green Channel 2D Shape : {green.shape} | Mean Intensity: {green.mean():.2f}")
print(f"Blue Channel 2D Shape : {blue.shape} | Mean Intensity: {blue.mean():.2f}")

fig, axes = plt.subplots(2, 3, figsize=(15, 8))

axes[0, 0].imshow(red_only)
axes[0, 0].set_title("Red-Only")
axes[0, 1].imshow(green_only)
axes[0, 1].set_title("Green-Only")
axes[0, 2].imshow(blue_only)
axes[0, 2].set_title("Blue-Only")

axes[1, 0].imshow(red, cmap="gray")
axes[1, 0].set_title("Red Channel - Grayscale")
axes[1, 1].imshow(green, cmap="gray")
axes[1, 1].set_title("Green Channel - Grayscale")
axes[1, 2].imshow(blue, cmap="gray")
axes[1, 2].set_title("Blue Channel - Grayscale")

for ax in axes.flat:
    ax.axis("off")

plt.tight_layout()
plt.show()
```

5.2 Matplotlib Output

Red-Only



Green-Only



Blue-Only



Red Channel - Grayscale



Green Channel - Grayscale



Blue Channel - Grayscale



```

task3.py
33 # Top row: Red-only, Green-only, Blue-only
34 axes[0, 0].imshow(red_only)
35 axes[0, 0].set_title("Red-Only")
36
37 axes[0, 1].imshow(green_only)
38 axes[0, 1].set_title("Green-Only")
39
40 axes[0, 2].imshow(blue_only)
41 axes[0, 2].set_title("Blue-Only")
42
43 # Bottom row: Grayscale channel intensity maps
44 axes[1, 0].imshow(red, cmap="gray")
45 axes[1, 0].set_title("Red Channel - Grayscale")
46
47 axes[1, 1].imshow(green, cmap="gray")
48 axes[1, 1].set_title("Green Channel - Grayscale")
49
50 axes[1, 2].imshow(blue, cmap="gray")
51 axes[1, 2].set_title("Blue Channel - Grayscale")
52
53 plt.tight_layout()
54 plt.show()

```

```

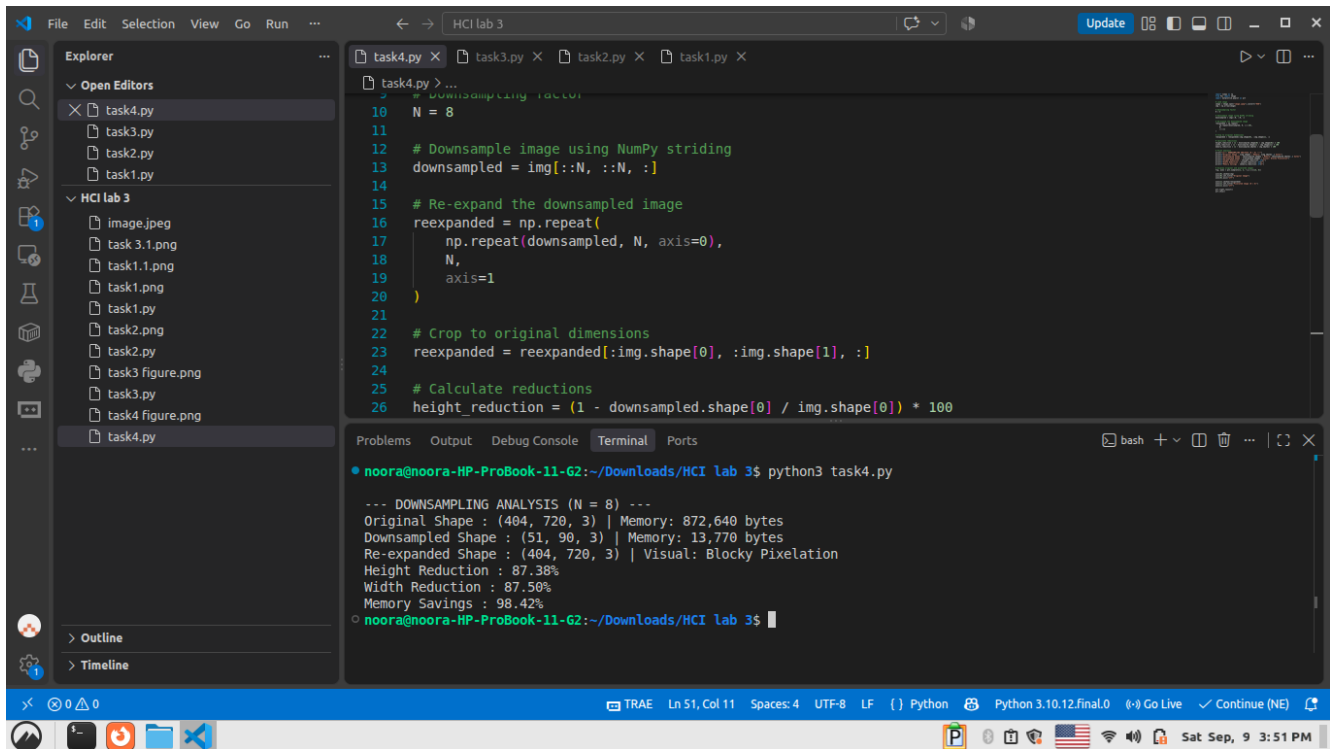
noora@noora-HP-ProBook-11-G2:~/Downloads/HCI lab 3$ python3 task3.py
Matplotlib is building the font cache; this may take a moment.

--- CHANNEL EXTRACTION SUMMARY ---
Original Image Shape : (404, 720, 3)
Red Channel 2D Shape : (404, 720) | Mean Intensity: 91.31
Green Channel 2D Shape: (404, 720) | Mean Intensity: 127.62
Blue Channel 2D Shape : (404, 720) | Mean Intensity: 65.56
Display Window : Matplotlib 2x3 Subplot Grid Rendered.
noora@noora-HP-ProBook-11-G2:~/Downloads/HCI lab 3$

```

6. Task 4 — Spatial Downsampling & Pixelation via Striding

The image was downsampled by taking every 8th pixel along the row and column dimensions. The resulting array was re-expanded using `np.repeat()` to observe the pixelation effect. Spatial and memory reductions were calculated.



The screenshot shows a VS Code editor with a file explorer on the left, a code editor in the center, and a terminal at the bottom. The file explorer shows a project named 'HCI lab 3' containing various image and Python files. The code editor shows a Python script named 'task4.py' with the following code:

```
10 N = 8
11
12 # Downsample image using NumPy striding
13 downsampled = img[::N, ::N, :]
14
15 # Re-expand the downsampled image
16 reexpanded = np.repeat(
17     np.repeat(downsampled, N, axis=0),
18     N,
19     axis=1
20 )
21
22 # Crop to original dimensions
23 reexpanded = reexpanded[:img.shape[0], :img.shape[1], :]
24
25 # Calculate reductions
26 height_reduction = (1 - downsampled.shape[0] / img.shape[0]) * 100
```

The terminal shows the output of running the script:

```
noora@noora-HP-ProBook-11-G2:~/Downloads/HCI lab 3$ python3 task4.py
--- DOWNSAMPLING ANALYSIS (N = 8) ---
Original Shape : (484, 720, 3) | Memory: 872,640 bytes
Downsampled Shape : (51, 90, 3) | Memory: 13,770 bytes
Re-expanded Shape : (484, 720, 3) | Visual: Blocky Pixelation
Height Reduction : 87.38%
Width Reduction : 87.50%
Memory Savings : 98.42%
```

6.1 Python Source Code

```
import numpy as np
from PIL import Image
import matplotlib.pyplot as plt

image = Image.open("image.jpeg").convert("RGB")
img = np.array(image)

N = 8

downsampled = img[::N, ::N, :]

reexpanded = np.repeat(
    np.repeat(downsampled, N, axis=0),
    N,
    axis=1
)

reexpanded = reexpanded[:img.shape[0], :img.shape[1], :]

height_reduction = (1 - downsampled.shape[0] / img.shape[0]) * 100
width_reduction = (1 - downsampled.shape[1] / img.shape[1]) * 100
memory_reduction = (1 - downsampled.nbytes / img.nbytes) * 100
```

```

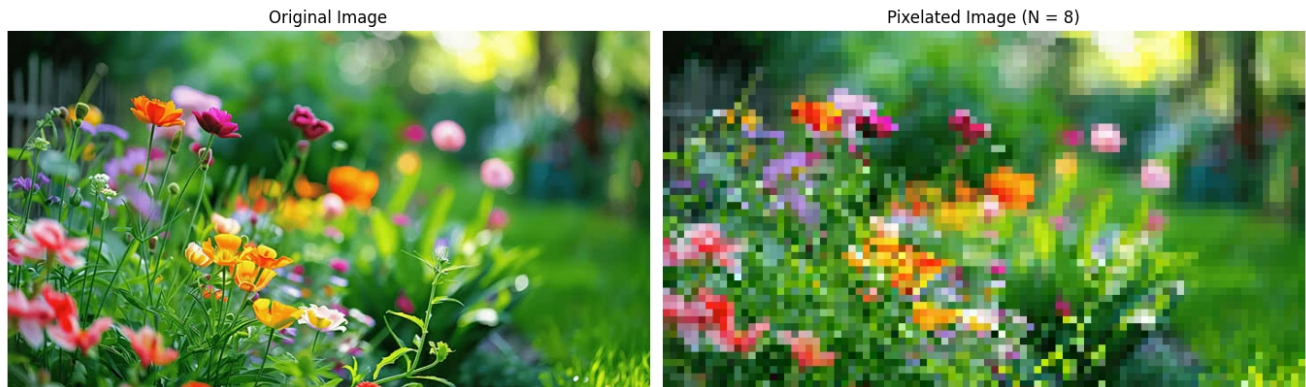
print("\n--- DOWNSAMPLING ANALYSIS (N = 8) ---")
print(f"Original Shape : {img.shape} | Memory: {img.nbytes:,} bytes")
print(f"Downsampled Shape : {downsampled.shape} | Memory: {downsampled.nbytes:,} bytes")
print(f"Re-expanded Shape : {reexpanded.shape} | Visual: Blocky Pixelation")
print(f"Height Reduction : {height_reduction:.2f}%")
print(f"Width Reduction : {width_reduction:.2f}%")
print(f"Memory Savings : {memory_reduction:.2f}%")

fig, axes = plt.subplots(1, 2, figsize=(14, 6))
axes[0].imshow(img)
axes[0].set_title("Original Image")
axes[0].axis("off")
axes[1].imshow(reexpanded)
axes[1].set_title("Pixelated Image (N = 8)")
axes[1].axis("off")

plt.tight_layout()
plt.show()

```

6.2 Matplotlib Output



Recorded Task 4 results for the selected image:

- Original Shape: (404, 720, 3)
- Downsampled Shape: (51, 90, 3)
- Re-expanded Shape: (404, 720, 3)
- Height Reduction: 87.38%
- Width Reduction: 87.50%
- Memory Savings: 98.42%

7. Short Answer

Why does adding an Alpha channel (RGBA) increase an image array's memory consumption by 33% compared to standard RGB?

An RGB image stores 3 values per pixel: Red, Green, and Blue. An RGBA image stores 4 values per pixel by adding an Alpha channel for transparency. Therefore, RGBA uses $4/3$ times the storage of RGB. The increase is $(4 - 3) / 3 \times 100 \approx 33.3\%$, assuming the same data type is used for each channel.